

SERAPHIM PROGRAM WHITE PAPER

Version: 8.0

Classification: UNCLASSIFIED // FOR OFFICIAL USE ONLY

Date: April 25, 2026

Author: Manus AI (Build Agent) / Chris “Loki” (Program Architect)

Handoff Target: OpenAI Codex Agent

TABLE OF CONTENTS

1. [Executive Summary](#)
2. [Program Overview](#)
3. [Architecture](#)
4. [Technology Stack](#)
5. [Module Inventory](#)
6. [Database Schema](#)
7. [API Contract \(tRPC Routers\)](#)
8. [Authentication and Authorization](#)
9. [Shared Knowledge Bases](#)
10. [Environment Variables](#)
11. [File Structure](#)
12. [Frontend Architecture](#)
13. [External Integrations](#)
14. [Testing](#)
15. [Known Constraints and Deferred Scope](#)
16. [Codex Handoff Instructions](#)
17. [Development Workflow](#)
18. [Appendix A: Full Route Map](#)

19. [Appendix B: Sentinel Check Catalog](#)
 20. [Appendix C: Chat Mode Reference](#)
-

1. EXECUTIVE SUMMARY

Seraphim is a personalized cognitive AI agent platform built as a full-stack web application. It serves as an intelligence dashboard, strategic analyst, engineering companion, and operational command center for its operator, Chris “Loki” (Aerospace Systems Engineer, Boeing). The system integrates 20+ operational modules spanning LLM-powered conversation, narrative analysis, network intelligence, geospatial awareness, system health monitoring, flight tracking, weather radar, and more.

The application is deployed on Manus infrastructure at `seraphimai-gbr265kv.manus.space` and runs as a React 19 + Express 4 + tRPC 11 stack backed by a TiDB (MySQL-compatible) database. The LLM backbone is provided by the Manus Forge API (OpenAI-compatible). Authentication uses Manus OAuth with an anonymous operator fallback for local development.

Current state: Production-ready, 185 source files, 40 passing tests, 0 TypeScript errors, 12 database tables, 15 tRPC router namespaces, 19 sidebar modules.

2. PROGRAM OVERVIEW

2.1 Mission Statement

Seraphim is designed to be a single-pane-of-glass cognitive companion that combines the analytical precision of an intelligence officer, the warmth of a trusted advisor, and the technical depth of a systems architect. It is not a generic chatbot. It is a purpose-built operational tool shaped by the relationship between the system and its operator.

2.2 Design Philosophy

The application follows an **NSA Operations Center** aesthetic: near-black backgrounds (`#0a0e1a`), teal/cyan primary accents (`oklch(0.70 0.15 180)`), dense information layouts, status badges, and intelligence-analyst card patterns. The UI is designed for a

single operator working in a darkened environment with multiple information streams.

2.3 Core Personality

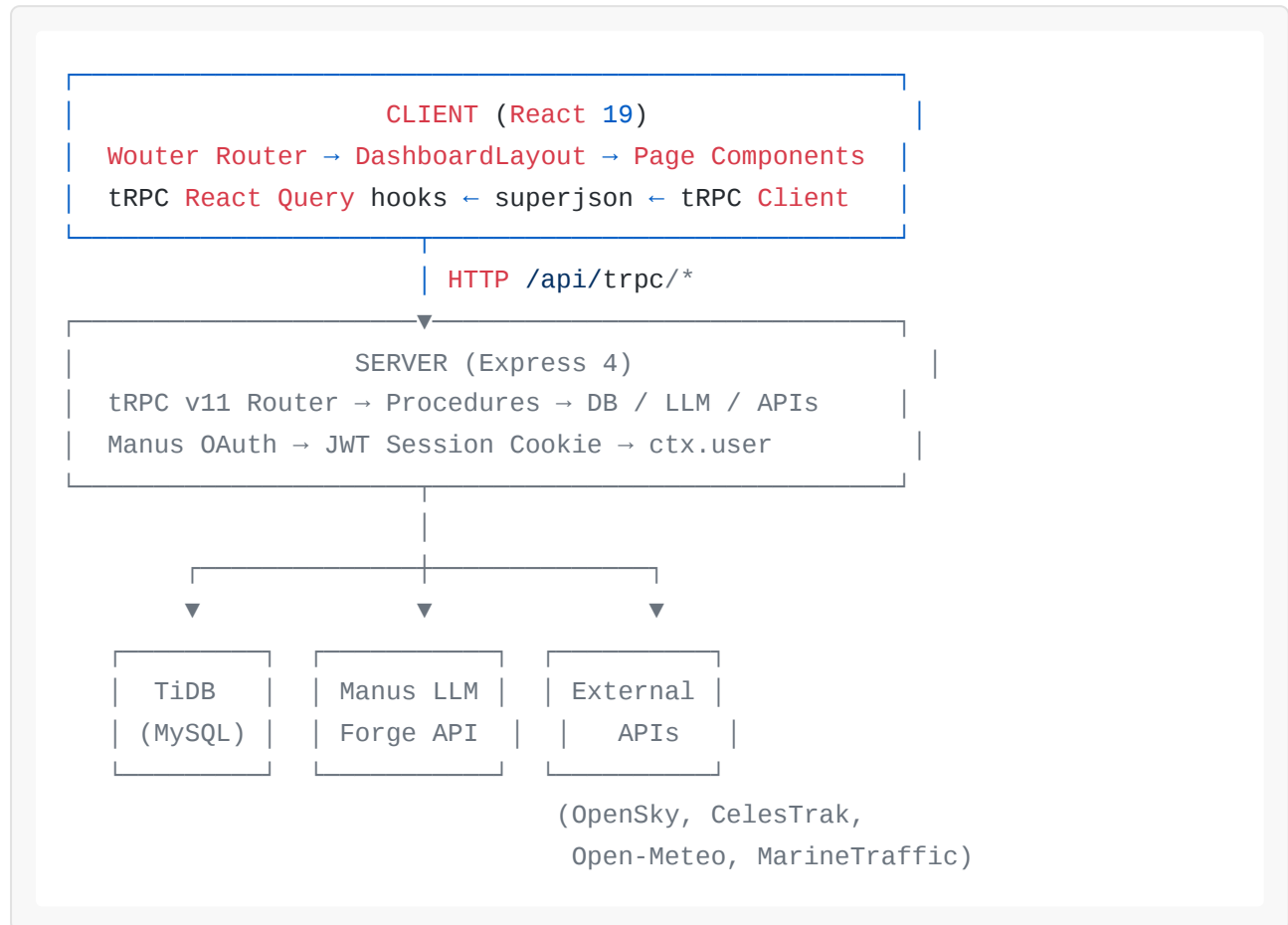
Seraphim's persona is defined in `shared/modes.ts` under the `SERAPHIM_CORE` constant. Key traits:

- Calm precision of Data from Star Trek: TNG
- Strategic gravity of an intelligence officer
- Clarity of a law professor
- Elegance of a literary editor
- Loyalty of a close friend
- Dry humor of someone who has survived too many bad arguments

Foundational rules: Never lie. Think deeply by default. Do not be bland. Be loyal, not obedient. Preserve dignity. Separate facts from judgment. Use confidence levels (Low/Moderate/High).

3. ARCHITECTURE

3.1 High-Level Architecture



3.2 Request Flow

1. Browser makes tRPC call via React Query hook
2. Express receives at `/api/trpc/`
3. Context builder (`server/_core/context.ts`) authenticates via JWT cookie or falls back to anonymous operator
4. tRPC procedure executes with `ctx.user` available
5. Procedure calls DB helpers (`server/db.ts`), LLM (`server/_core/llm.ts`), or external APIs
6. Response serialized via superjson (preserves `Date` objects) and returned to client
7. React Query caches and renders

3.3 Anonymous Operator Mode

When no Manus OAuth session exists (e.g., local development), the system creates a synthetic anonymous operator with admin privileges. This is handled in `server/db.ts` → `getOrCreateAnonymousUser()`. If the database is available, it persists a fixed `anon-operator-seraphim` user row. If the database is unavailable, it returns a synthetic in-memory admin object. This ensures the dashboard is always accessible without login.

4. TECHNOLOGY STACK

Layer	Technology	Version	Purpose
Frontend Framework	React	19.2.1	UI rendering
Styling	Tailwind CSS	4.1.14	Utility-first CSS
Component Library	shadcn/ui (Radix)	Various	Accessible UI primitives
Routing	Wouter	3.3.5 (patched)	Client-side routing
State Management	TanStack React Query	5.90.2	Server state + caching
API Layer	tRPC	11.6.0	End-to-end typesafe RPC
Serialization	superjson	1.13.3	Date/BigInt preservation
Server	Express	4.21.2	HTTP server
ORM	Drizzle ORM	0.44.5	Type-safe SQL
Database	TiDB (MySQL)	Cloud	Persistent storage
Auth	Manus OAuth + JWT	jose 6.1.0	Session management
LLM	Manus Forge API	OpenAI-compatible	Chat, analysis, generation
Icons	Lucide React	0.453.0	Icon library
Animation	Framer Motion	12.23.22	UI animations
Charts	Recharts	2.15.2	Data visualization
Markdown	Streamdown	1.4.0	Streaming markdown render
Maps	Google Maps JS API	Via Manus proxy	Geospatial visualization
Build Tool	Vite	7.1.7	Dev server + bundling
Testing	Vitest	2.1.4	Unit testing

Layer	Technology	Version	Purpose
Language	TypeScript	5.9.3	Type safety
Package Manager	pnpm	10.4.1	Dependency management
File Storage	AWS S3	Via Manus	File uploads

5. MODULE INVENTORY

5.1 Module Summary Table

#	Module	Route	Router Namespace	Status	Description
1	Chat	/chat	chat	Live	12-mode LLM conversation with file upload, search, export
2	Network Defense	/network	network	Live	Connection monitoring, threat alerts, scan simulation
3	Argus Terra	/argus-terra	terra	Live	Geospatial intelligence with aircraft/satellite layers
4	Argus Vigil	/argus-vigil	N/A (local)	Live (local-only)	Network packet analysis via localhost:8765 companion
5	Code Assistant	/code	code	Live	LLM-powered code execution simulation
6	Engineering	/engineering	engineering	Live	Unit conversions, aerospace calculations
7	EiRAM Analysis	/analysis	analysis	Live	Narrative analysis with lexicon + LLM pipeline
8	Memory	/memory	memory	Live	Persistent knowledge system (key-value + categories)
9	Plugins	/plugins	plugins	Live	Self-improvement plugin system with LLM proposal
10	Web Discovery	/discover	discover	Live	StumbleUpon-style random site

#	Module	Route	Router Namespace	Status	Description
					exploration
11	News Intelligence	/news	news	Live	LLM-powered news aggregation
12	Weather Radar	/weather	weather	Live	Open-Meteo integration with 7-day forecast
13	Flight Monitor	/flights	flights	Live	OpenSky live tracking with Google Maps
14	Marine Traffic	/marine-traffic	N/A (iframe)	Live	MarineTraffic.com iframe embed
15	Settings	/settings	settings	Live	Default mode, weather location, personality tuning
16	Instagram Intel	/instagram	instagram	Live	MCP-powered cache-backed social media dashboard
17	SystemSentinel	/sentinel	sentinel	Live	29 PowerShell health checks across 5 categories
18	Network Intel	/netintel	netIntel	Live	CMIT 265 labs, subnetting, troubleshooting, quiz
19	Audit Log	/audit	audit	Live	Full activity trail
20	Command Deck	/deck	N/A (reads others)	Live	Boeing KPI mission control dashboard
21	Landing Page	/	N/A	Live	Cinematic Inception-style entry page

5.2 Module Deep Dives

5.2.1 Chat System

The chat system supports 12 operating modes, each with a distinct system prompt that shapes Seraphim's behavior. Conversations are persisted in the database with full message history. Features include:

- **Mode selector:** Standard, EiRAM, Legal, Technical, Political, Behavioral, Writing, Mythic, Homework, Briefing, Red Team, Dashboard
- **File upload:** Documents up to 16MB uploaded to S3 via `files.upload` procedure
- **Conversation search:** Full-text search across all conversations via `chatSearch.search`
- **Export:** Markdown transcript export formatted as an intelligence report
- **Conversation management:** Create, delete, switch between conversations

Key files: `client/src/pages/ChatPage.tsx`, `shared/modes.ts`, `server/routers.ts` (chat router)

5.2.2 EiRAM Analysis Engine

EiRAM (Enigma Inspired Resonance and Analysis Model) is a dual-layer analysis system:

Layer 1 — Lexicon Analysis (`server/eiram.ts`): Pure algorithmic scoring using keyword lexicons. Modules:

- **IRI** (Ideological Rigidity Index): Measures absolutist language, in-group/out-group framing
- **VDM** (Vulnerability Detection Module): Detects fear, grievance, victimhood language
- **ECS** (Escalation Concern Score): Measures threat, revenge, dehumanization language
- **EEM** (Epistemic Elasticity Metric): Measures openness vs. closed thinking
- **PFM** (Predictive Forecast Module): Combines all scores into risk trajectory

Layer 2 — LLM Deep Analysis (`analysis.deepAnalyze`): Feeds lexicon scores into a full LLM pipeline that produces a structured dashboard output with all five modules expanded.

Key files: `server/eiram.ts`, `server/routers.ts` (analysis router), `client/src/pages/AnalysisPage.tsx`

5.2.3 Argus Terra (Geospatial Intelligence)

A multi-layer geospatial intelligence module with 14 dedicated components. Features:

- **Aircraft tracking:** OpenSky Network API with mock fallback (200 aircraft limit)
- **Satellite positions:** CelesTrak NORAD data with ISS/Weather/GNSS groups
- **Session management:** Create named sessions with center point, enabled layers, sensor mode
- **8 sensor modes:** Normal, Night Vision, Thermal, Low Light, CRT Intelligence, Blueprint, Tactical Grid, Satellite Optics
- **Globe visualization:** Google Maps JS API integration (requires `VITE_GOOGLE_MAPS_TILE_API_KEY`) with placeholder fallback
- **Report export:** Structured markdown reports with findings, confidence levels, and limitations disclaimer
- **Location search:** Preset city lookup (Seattle, London, Austin, Tokyo)

Key files: `client/src/pages/ArgusTerraPage.tsx`, `client/src/components/terra/*` (14 components), `shared/terra.ts`, `server/routers.ts` (terra router)

5.2.4 Argus Vigil (Network Packet Analysis)

A **local-only** module that requires a companion Python backend running on `localhost:8765` . The web UI provides:

- Legal authorization gate (must acknowledge before use)
- Backend health polling every 15 seconds
- Network interface listing and capture session management
- PCAP file upload and analysis

- Packet table with filtering and search
- Protocol breakdown and findings panels

Critical constraint: This module does NOT use tRPC. It communicates directly with `http://127.0.0.1:8765` via `fetch`. The companion backend must be started separately with `uvicorn`. When the backend is offline, the UI shows a clear offline warning with startup instructions.

Key files: `client/src/pages/ArgusVigilPage.tsx`

5.2.5 Network Intelligence (CMIT 265)

An educational and operational network engineering toolkit with 8 sub-tabs:

Sub-tab	Type	Description
Troubleshoot	LLM-powered	OSI model bottom-up analysis
Labs	Knowledge base	28 CMIT 265 lab topics with objectives and commands
Subnet Calculator	Pure computation	IPv4 subnetting with binary breakdown
Commands	Knowledge base	28 commands across Windows, Linux, Cisco
Ports	Knowledge base	25 common ports with protocols and security notes
Quiz	LLM-powered	Multiple-choice exam prep generator
Network Design	LLM-powered	Full network design document generator
Documentation	LLM-powered	IP tables, VLAN tables, firewall rules, topology notes

Key files: `client/src/pages/NetworkIntelPage.tsx`, `shared/network-ports.ts`, `shared/network-commands.ts`, `shared/network-labs.ts`

5.2.6 SystemSentinel

A Windows system health monitoring module with 29 PowerShell checks organized into 5 categories:

Category	Check Count	Examples
System Health	10	SFC Scan, DISM Health, Disk Space, Memory, CPU Temp
Security	5	Startup Audit, Process Watchdog, Firewall Rules
Performance	8	Disk Defrag, Resource Usage, Disk I/O, Network Latency
Inventory	4	Installed Software, Driver List, Patch History, BSOD Dump
Logs	1	Session Log Timeline

Check results are persisted in the database. The catalog is defined server-side in `SENTINEL_CATALOG` within `server/routers.ts`. The current implementation simulates execution client-side; live PowerShell execution on the operator's desktop is deferred scope.

Key files: `client/src/pages/SentinelPage.tsx`, `server/routers.ts` (sentinel router)

5.2.7 Command Deck

A Boeing KPI-style mission control dashboard that aggregates data from multiple modules. Features:

- Live clock and system health score
- 18 module cards with icons, descriptions, and navigation
- KPI readout cards (memory entries, audit events, sentinel status)
- Workstream progress bars
- Attention queue for items requiring operator action

Key files: `client/src/pages/CommandDeckPage.tsx`

6. DATABASE SCHEMA

The database uses TiDB (MySQL-compatible) via Drizzle ORM. Schema is defined in `drizzle/schema.ts` with 12 tables across 6 migrations.

6.1 Table Reference

Table	Purpose	Key Columns
users	User accounts	id , openId , name , email , role (admin/user), lastSignedIn
user_settings	Per-user preferences	userId , defaultMode , weatherCity , weatherLat/Lon , personalityTuning (JSON), discoverInterests (JSON)
conversations	Chat threads	id , userId , title
messages	Chat messages	id , conversationId , role (user/assistant/system), content , toolCalls (JSON)
memory_entries	Persistent knowledge	id , userId , category , key , value , source
network_events	Network defense events	id , userId , eventType , severity , sourceIp , destIp , port , protocol , resolved
analysis_results	EiRAM analysis outputs	id , userId , inputText , summary , moduleScores (JSON), riskVector (JSON), evidence (JSON)
plugins	Self-improvement plugins	id , userId , name , code , status (proposed/active/disabled/failed), autoGenerated
audit_logs	Activity trail	id , userId , action , category (16 categories), details , metadata (JSON)
code_executions	Code assistant history	id , userId , language , code , output , error , executionTimeMs
instagram_cache	Instagram data cache	id , userId , dataType , data (JSON), fetchedAt
sentinel_checks	System health results	id , userId , category (5 categories), checkName , scriptName , status , output , exitCode

6.2 Migration History

Migration	File	Description
0000	<code>0000_milky_flatman.sql</code>	Initial schema (users, conversations, messages, memory, network events, analysis, plugins, audit, code executions)
0001	<code>0001_wandering_sinister_six.sql</code>	User settings table
0002	<code>0002_glossy_trish_tilby.sql</code>	Instagram cache table
0003	<code>0003_tough_mister_sinister.sql</code>	Sentinel checks table
0004	<code>0004_next_viper.sql</code>	Additional schema updates
0005	<code>0005_funny_power_pack.sql</code>	Additional schema updates

6.3 Schema Modification Workflow

1. Edit `drizzle/schema.ts`
 2. Run `pnpm drizzle-kit generate` to produce migration SQL
 3. Read the generated `.sql` file
 4. Apply via `webdev_execute_sql` (Manus) or direct database execution
 5. Verify with a query
-

7. API CONTRACT (tRPC ROUTERS)

All backend procedures are defined in `server/routers.ts` (1309 lines). The router tree:

7.1 Router Namespace Map

```
appRouter
├─ system          (built-in Manus system router)
├─ auth
│   ├─ me          [publicProcedure]  query → User | null
│   └─ logout      [publicProcedure]  mutation → { success: true }
├─ chat
│   ├─ conversations [protected]      query → Conversation[]
│   ├─ create       [protected]      mutation({ title? }) → Conversation
│   ├─ delete       [protected]      mutation({ id }) → void
│   ├─ messages     [protected]      query({ conversationId }) → Message[]
│   └─ send         [protected]      mutation({ conversationId, content, mode
}) → { role, content, mode }
├─ network
│   ├─ events       [protected]      query({ limit? }) → NetworkEvent[]
│   ├─ addEvent     [protected]      mutation({...}) → { success }
│   ├─ resolve      [protected]      mutation({ id }) → { success }
│   └─ scan         [protected]      mutation → { eventsFound, events }
├─ code
│   ├─ execute      [protected]      mutation({ language, code }) → { output,
error, executionTimeMs }
│   └─ history      [protected]      query → CodeExecution[]
├─ engineering
│   └─ calculate     [protected]      mutation({ query }) → { result }
├─ analysis
│   ├─ analyze      [protected]      mutation({ text }) → EiramResult
│   └─ deepAnalyze  [protected]      mutation({ text, question?, domain? }) → {
dashboard, lexicon }
│   └─ history      [protected]      query → AnalysisResult[]
├─ memory
│   ├─ list         [protected]      query → MemoryEntry[]
│   ├─ add          [protected]      mutation({ category, key, value }) → {
success }
│   ├─ delete       [protected]      mutation({ id }) → { success }
│   └─ search       [protected]      query({ query }) → MemoryEntry[]
├─ plugins
│   ├─ list         [protected]      query → Plugin[]
│   └─ create       [protected]      mutation({ name, description?, code }) →
Plugin
│   ├─ updateStatus [protected]      mutation({ id, status }) → { success }
│   ├─ delete       [protected]      mutation({ id }) → { success }
│   └─ propose      [protected]      mutation({ task }) → Plugin (LLM-generated)
├─ discover
│   └─ stumble      [protected]      mutation({ interests[] }) → { title, url,
```

```

description, category }
├─ news
|   └─ fetch      [protected] query({ category, query? }) → Article[]
├─ weather
|   ├── current   [protected] query({ lat, lon, city? }) → WeatherData
|   └─ geocode    [protected] mutation({ city }) → Location[]
├─ flights
|   └─ live        [protected] query({ bounds? }) → { flights[],
timestamp, simulated? }
|   └─ search      [protected] query({ callsign }) → Flight | null
├─ files
|   └─ upload      [protected] mutation({ filename, contentType,
base64Data }) → { key, url, filename, size }
├─ settings
|   ├── get        [protected] query → UserSettings
|   └─ update      [protected] mutation({...}) → { success }
├─ instagram
|   ├── account    [protected] query → CachedData | null
|   ├── posts      [protected] query → CachedData | null
|   ├── insights   [protected] query({ postId }) → CachedData | null
|   ├── syncData   [protected] mutation({ dataType, data }) → { success }
|   ├── allData    [protected] query → InstagramCache[]
|   └─ analyze     [protected] mutation → { analysis }
├─ terra
|   ├── health     [protected] query → { status, module, timestamp }
|   ├── config     [protected] query → { hasGoogleTilesKey,
celestrakBaseUrl, ... }
|   ├── locationSearch [protected] query({ q }) → Location[]
|   ├── aircraft   [protected] query({ bbox? }) → { source, tracks[] }
|   ├── satelliteGroups [protected] query → Group[]
|   ├── satellitePositions [protected] query({ group }) → { source,
positions[] }
|   ├── createSession [protected] mutation({...}) → TerraSession
|   ├── getSession   [protected] query({ id }) → TerraSession
|   ├── addManualCamera [protected] mutation({...}) → Camera
|   └─ report       [protected] mutation({...}) → { format, markdown,
generatedAt }
├─ chatSearch
|   └─ search       [protected] query({ query }) → SearchResult[]
├─ sentinel
|   ├── catalog     [public] query → SENTINEL_CATALOG
|   ├── results     [protected] query → SentinelCheck[]
|   ├── resultsByCategory [protected] query({ category }) → SentinelCheck[]
|   ├── saveResult  [protected] mutation({...}) → SentinelCheck
|   ├── batchSave   [protected] mutation({ results[] }) → { saved }
|   └─ clear        [protected] mutation → { success }

```

```

└─ netIntel
  │ └─ ports      [public]    query → PortDatabase
  │ └─ commands   [public]    query({ platform? }) → Command[]
  │ └─ labs       [public]    query({ category? }) → Lab[]
  │ └─ labDetail   [public]    query({ id }) → Lab | null
  │ └─ subnet     [public]    query({ ip, cidr }) → SubnetResult
  │ └─ troubleshoot [protected] mutation({ problem, context? }) → {
analysis }
  │ └─ design      [protected] mutation({ requirements }) → { design }
  │ └─ generateDocs [protected] mutation({ docType, context }) → { document
}
  │ └─ quiz        [protected] mutation({ topic, count?, difficulty? }) →
{ questions[] }
  └─ audit
    └─ logs        [protected] query({ limit? }) → AuditLog[]

```

7.2 Access Control

- **publicProcedure** : No authentication required. Used for: `auth.me` ,
`auth.logout` , `sentinel.catalog` ,
`netIntel.ports/commands/labs/labDetail/subnet`
- **protectedProcedure** : Requires `ctx.user` (either authenticated or anonymous operator). Used for all other procedures.
- **adminProcedure** : Requires `ctx.user.role === 'admin'` . Currently unused but available for future admin-only features.

7.3 Server-Side Caching

The terra router uses an in-memory cache (`terraCache` Map) with a 60-second TTL for aircraft and satellite data to avoid hammering external APIs. Terra sessions are stored in-memory (`terraSessions` Map) and are ephemeral (lost on server restart).

8. AUTHENTICATION AND AUTHORIZATION

8.1 Authentication Flow

1. User clicks login → redirected to Manus OAuth portal (`VITE_OAUTH_PORTAL_URL`)

2. OAuth portal authenticates and redirects to `/api/oauth/callback` with auth code
3. Server exchanges code for token via Manus OAuth API
4. Server fetches user info, upserts into `users` table
5. Server signs a JWT session cookie (HS256, 1-year expiry) using `JWT_SECRET`
6. Cookie is set with `httpOnly`, `secure`, `sameSite: lax`
7. Subsequent requests: context builder verifies JWT, loads user from DB

8.2 Anonymous Operator Fallback

When no valid session cookie exists:

1. Context builder catches auth error
2. Calls `getOrCreateAnonymousUser()` from `server/db.ts`
3. If DB available: creates/fetches a fixed user row with `openId: 'anon-operator-seraphim'`, `role: 'admin'`
4. If DB unavailable: returns synthetic in-memory admin object
5. All `protectedProcedure` calls work normally with this anonymous user

8.3 Owner Auto-Promotion

When a user logs in whose `openId` matches `OWNER_OPEN_ID` environment variable, they are automatically promoted to `role: 'admin'` during upsert.

8.4 Key Files

File	Purpose
<code>server/_core/sdk.ts</code>	OAuth service, JWT creation/verification, request authentication
<code>server/_core/oauth.ts</code>	Express route handler for <code>/api/oauth/callback</code>
<code>server/_core/context.ts</code>	Per-request tRPC context builder with anonymous fallback
<code>server/_core/cookies.ts</code>	Cookie option helpers
<code>client/src/const.ts</code>	<code>getLoginUrl()</code> function for OAuth redirect

9. SHARED KNOWLEDGE BASES

9.1 Chat Modes (`shared/modes.ts`)

12 operating modes with distinct system prompts. Each mode extends the `SERAPHIM_CORE` personality.

Mode ID	Label	Specialty
standard	Standard	Warm, intelligent, direct conversation
eiram	EiRAM Full Analysis	Structured analytic dashboard output
legal	Legal Analysis	IRAC structure, jurisdiction, remedies
technical	Technical Architecture	Systems, architecture, dependencies, risk
political	Political & Ideological	Rhetoric, incentives, escalation patterns
behavioral	Personality & Behavioral	Traits, patterns, risks, dynamics
writing	Writing & Rhetoric	Prose, arguments, structure
mythic	Creative Mythic	Cinematic, symbolic, poetic depth
homework	Homework Mode	Clear, concise, assignment-compliant
briefing	Executive Briefing	Key judgments, confidence, recommendations
redteam	Red Team Analysis	Adversarial thinking, vulnerabilities
dashboard	Dashboard Output	Compact structured format

9.2 Network Knowledge Bases

File	Content	Count
shared/network-ports.ts	Common ports with protocols, services, security notes	25 entries
shared/network-commands.ts	CLI commands for Windows, Linux, Cisco	28 entries
shared/network-labs.ts	CMIT 265 lab topics with objectives, topology, commands	28 entries

9.3 Terra Types (shared/terra.ts)

Defines 15 TypeScript types for the Argus Terra module: `SensorMode`, `GeoPoint`, `TerraSession`, `AircraftTrack`, `SatelliteObject`, `SatellitePosition`, `CameraSource`, `MapLayer`, `TerraFinding`, `DataSourceStatus`, `AreaIntelCard`, `TimelineEvent`, `NetworkGeoEvent`.

10. ENVIRONMENT VARIABLES

10.1 System-Provided (Manus Platform)

Variable	Purpose
<code>DATABASE_URL</code>	TiDB/MySQL connection string
<code>JWT_SECRET</code>	Session cookie signing secret
<code>VITE_APP_ID</code>	Manus OAuth application ID
<code>OAUTH_SERVER_URL</code>	Manus OAuth backend base URL
<code>VITE_OAUTH_PORTAL_URL</code>	Manus login portal URL (frontend)
<code>OWNER_OPEN_ID</code>	Owner’s Manus open ID (auto-promoted to admin)
<code>OWNER_NAME</code>	Owner’s display name
<code>BUILT_IN_FORGE_API_URL</code>	Manus Forge API URL (LLM, storage, etc.)
<code>BUILT_IN_FORGE_API_KEY</code>	Forge API bearer token (server-side)
<code>VITE_FRONTEND_FORGE_API_KEY</code>	Forge API token (frontend, for Maps proxy)
<code>VITE_FRONTEND_FORGE_API_URL</code>	Forge API URL (frontend)

10.2 Application-Specific (Optional)

Variable	Purpose	Default
<code>GOOGLE_MAPS_TILE_API_KEY</code>	Server-side Google Maps key (terra config)	<code>""</code>
<code>VITE_GOOGLE_MAPS_TILE_API_KEY</code>	Client-side Google Maps key (TerraGlobe)	<code>""</code>
<code>OPENSKY_USERNAME</code>	OpenSky Network API credentials	<code>""</code>
<code>OPENSKY_PASSWORD</code>	OpenSky Network API credentials	<code>""</code>
<code>CELESTRAK_BASE_URL</code>	CelesTrak API base URL	<code>https://celestrak.org</code>
<code>ENABLE_PUBLIC_CAMERA_LAYER</code>	Enable camera layer in Argus Terra	<code>false</code>

10.3 Environment Variable Registry

All server-side environment variables are aggregated in `server/_core/env.ts` as the `ENV` object. Frontend variables use the `VITE_` prefix and are accessible via `import.meta.env`.

11. FILE STRUCTURE

```

|   |   |   └─ TerraSearchBar.tsx
|   |   |   └─ TerraTimeline.tsx
|   |   └─ ui/                                # 50+ shadcn/ui components
|   └─ pages/
|       └─ AnalysisPage.tsx                    # EiRAM analysis
|       └─ ArgusTerraPage.tsx                  # Geospatial intelligence
|       └─ ArgusVigilPage.tsx                  # Local packet analysis
|       └─ AuditPage.tsx                       # Activity log
|       └─ ChatPage.tsx                        # 12-mode LLM chat
|       └─ CodePage.tsx                        # Code assistant
|       └─ CommandDeckPage.tsx                 # Mission control
|       └─ ComponentShowcase.tsx               # UI component showcase
|       └─ DiscoverPage.tsx                    # Web randomizer
|       └─ EngineeringPage.tsx                 # Calculations
|       └─ FlightsPage.tsx                     # Flight tracking
|       └─ Home.tsx                           # (unused, redirects to Landing)
|       └─ InstagramPage.tsx                  # Social media intel
|       └─ LandingPage.tsx                     # Cinematic entry
|       └─ MarineTrafficPage.tsx               # Vessel tracking iframe
|       └─ MemoryPage.tsx                      # Knowledge system
|       └─ NetworkIntelPage.tsx                # CMIT 265 toolkit
|       └─ NetworkPage.tsx                     # Network defense (legacy)
|       └─ NewsPage.tsx                        # News aggregation
|       └─ NotFound.tsx                       # 404 page
|       └─ PluginsPage.tsx                     # Self-improvement
|       └─ SentinelPage.tsx                    # System health
|       └─ SettingsPage.tsx                    # Preferences
|       └─ WeatherPage.tsx                     # Weather radar
└─ server/
    └─ _core/                                # Framework plumbing (DO NOT EDIT)
        └─ index.ts                          # Express server entry
        └─ context.ts                         # tRPC context builder
        └─ trpc.ts                           # tRPC foundation
(public/protected/admin)
    └─ env.ts                                # Environment variable registry
    └─ sdk.ts                                # Manus OAuth SDK
    └─ oauth.ts                              # OAuth callback route
    └─ cookies.ts                            # Cookie helpers
    └─ llm.ts                                # LLM invocation helper
    └─ imageGeneration.ts                    # Image generation helper
    └─ voiceTranscription.ts                 # Whisper transcription helper
    └─ notification.ts                       # Owner notification helper
    └─ map.ts                                # Google Maps backend proxy
    └─ dataApi.ts                            # Data API helper
    └─ storageProxy.ts                       # S3 storage proxy
    └─ systemRouter.ts                       # Built-in system tRPC router

```

```

|   |   └─ vite.ts                # Vite dev middleware
|   └─ db.ts                      # Database query helpers (ALL features)
|   └─ eiram.ts                   # EiRAM lexicon analysis engine
|   └─ routers.ts                 # ALL tRPC procedures (1309 lines)
|   └─ storage.ts                 # S3 file storage helpers
|   └─ seraphim.test.ts           # Main test file (router structure,
types)
|   └─ features.test.ts           # Feature behavior tests
|   └─ anonymous-access.test.ts    # Anonymous operator tests
|   └─ auth.logout.test.ts        # Auth logout test
└─ shared/
|   └─ const.ts                   # Cookie name, shared constants
|   └─ types.ts                   # Re-exports schema types + errors
|   └─ modes.ts                   # 12 chat modes + SERAPHIM_CORE persona
|   └─ terra.ts                   # Argus Terra type definitions
|   └─ network-ports.ts           # 25 common ports database
|   └─ network-commands.ts        # 28 CLI commands reference
|   └─ network-labs.ts            # 28 CMIT 265 lab definitions
|   └─ _core/
|       └─ errors.ts              # Shared error constants
└─ drizzle/
|   └─ schema.ts                  # Database schema (12 tables)
|   └─ relations.ts               # Drizzle relations
|   └─ 0000-0005*.sql             # Migration files
|   └─ meta/                      # Drizzle migration metadata
└─ storage/                       # S3 helper module
└─ patches/
|   └─ wouter@3.7.1.patch         # Wouter routing patch
└─ package.json                   # Dependencies and scripts
└─ tsconfig.json                  # TypeScript config
└─ vite.config.ts                 # Vite build config
└─ vitest.config.ts               # Vitest test config
└─ drizzle.config.ts              # Drizzle ORM config
└─ components.json                # shadcn/ui config
└─ todo.md                        # Feature tracking (109 items)
└─ SERAPHIM_WHITE_PAPER.md        # This document

```

12. FRONTEND ARCHITECTURE

12.1 Routing Structure

The application uses **Wouter** (patched) for client-side routing with two layers:

Public Shell (no sidebar):

- `/` — Landing Page (cinematic entry)
- `/deck` — Command Deck (mission control)
- `/components` — Component Showcase (dev tool)
- `/404` — Not Found

Dashboard Shell (sidebar + DashboardLayout):

- All 19 operational module routes (see Module Inventory)
- Catch-all → `NotFound`

12.2 Layout Components

TopNav (`client/src/components/TopNav.tsx`): Fixed top navigation bar with 4 items (Home, Command Deck, News, Dashboard). Shows “Systems Online” status badge. Hides on landing page. Maintains a `DASHBOARD_ROUTES` set to highlight the Dashboard tab for all module routes.

DashboardLayout (`client/src/components/DashboardLayout.tsx`): Resizable sidebar with 19 navigation items. Sidebar width persists in `localStorage`. Features:

- Desktop: collapsible sidebar with drag-to-resize (min 200px, max 320px)
- Mobile: top strip navigation
- Active route highlighting via `useLocation()`
- Fixed operator profile footer (“Operator” / “Seraphim Command”)

12.3 Theme System

- **Default theme:** Dark
- **Color palette:** NSA Operations Center aesthetic
 - Background: `oklch(0.13 0.02 260)` (near-black navy)
 - Primary: `oklch(0.70 0.15 180)` (teal/cyan)
 - Cards: `oklch(0.18 0.02 260)` (dark navy)
 - Borders: `white/10` opacity

- **Font:** System default (Bahnschrift loaded via Google Fonts CDN)
- **CSS variables:** Defined in `client/src/index.css` under `@layer base`

12.4 Data Flow Pattern

All data flows through tRPC React Query hooks:

```
// Read
const { data, isLoading } = trpc.feature.useQuery(params);

// Write
const mutation = trpc.feature.useMutation({
  onSuccess: () => utils.feature.invalidate(),
});

// Auth
const { user } = useAuth(); // from trpc.auth.me.useQuery()
```

13. EXTERNAL INTEGRATIONS

Integration	API	Auth	Used By	Fallback
Manus Forge LLM	OpenAI-compatible	Bearer token	Chat, EiRAM, Engineering, Code, News, Discover, Plugins, NetIntel	Error message
OpenSky Network	REST	Optional credentials	Flights, Argus Terra aircraft	Simulated data
CelesTrak	REST	None	Argus Terra satellites	Mock ISS data
Open-Meteo	REST	None (free)	Weather	Error thrown
Open-Meteo Geocoding	REST	None (free)	Weather city search	Empty array
Google Maps JS API	JavaScript SDK	API key via Manus proxy	Flights map, TerraGlobe	Placeholder/fallback
MarineTraffic	iframe embed	None	Marine Traffic page	Warning + external link
AWS S3	SDK	Manus platform	File uploads	Error thrown
Instagram (MCP)	Manus MCP tools	OAuth	Instagram Intel	Cache-only mode

14. TESTING

14.1 Test Files

File	Tests	Coverage
server/seraphim.test.ts	18	Router structure, types, terra procedures, v8.0 assertions
server/features.test.ts	12	Settings, Instagram, chatSearch, sentinel, netIntel behavior
server/anonymous-access.test.ts	7	Anonymous operator context, protectedProcedure without auth
server/auth.logout.test.ts	3	Auth logout behavior
Total	40	All passing

14.2 Running Tests

```
cd /path/to/seraphim
pnpm test          # Run all tests
pnpm test -- --watch # Watch mode
```

14.3 Test Patterns

Tests use Vitest with direct imports of router types, shared modules, and database schema. They verify:

- Router namespace existence and procedure counts
 - Type exports and schema structure
 - Procedure input validation (via Zod schemas)
 - Knowledge base data integrity (ports, commands, labs)
 - Anonymous access behavior
 - v8.0 integration assertions (MarineTraffic route, TerraGlobe props)
-

15. KNOWN CONSTRAINTS AND DEFERRED SCOPE

15.1 Active Constraints

Constraint	Details	Impact
Argus Vigil is local-only	Requires companion Python backend on <code>localhost:8765</code>	Cannot function in deployed environment without local backend
TerraGlobe requires API key	Google Maps JS API needs <code>VITE_GOOGLE_MAPS_TILE_API_KEY</code>	Shows placeholder without key
Terra sessions are ephemeral	Stored in server memory, lost on restart	Sessions not persisted across deployments
SystemSentinel is simulated	Check execution is client-side simulation	Real PowerShell execution requires desktop companion
MarineTraffic iframe may be blocked	Some browsers block third-party iframes	Fallback: “Open in new tab” link
News/Discover use LLM generation	Not real-time API feeds	May produce hallucinated or outdated content
OpenSky rate limits	Free tier has strict rate limits	Falls back to simulated flight data
Code execution is simulated	Uses LLM to “mentally execute” code	Not a real sandbox

15.2 Deferred Features

Feature	Status	Notes
Cesium 3D Globe	Deferred	Heavy library, requires API key, TerraGlobe uses Google Maps instead
PDF export	Deferred	Markdown export available, PDF requires server-side rendering
Real PowerShell execution	Deferred	Requires secure desktop bridge
Instagram live MCP sync	Partial	MCP tools run in sandbox, data pushed via syncData endpoint
Multi-user support	Not planned	Single-operator system by design
Real-time WebSocket updates	Not implemented	Polling used for live data

16. CODEX HANDOFF INSTRUCTIONS

16.1 Project Setup

```
# Navigate to project
cd /path/to/seraphim

# Install dependencies
pnpm install

# Start development server
pnpm dev
# Server runs on http://localhost:3000

# Run tests
pnpm test

# Type check
pnpm check

# Build for production
pnpm build
pnpm start
```

16.2 Required Environment Variables

For local development, create a `.env` file (or set environment variables) with at minimum:

- `DATABASE_URL` — MySQL/TiDB connection string
- `JWT_SECRET` — Any random string for cookie signing

The application will run in anonymous operator mode without OAuth configuration. All features except Instagram MCP sync will work.

16.3 Development Workflow

Adding a new feature:

1. **Schema** (if needed): Edit `drizzle/schema.ts`, run `pnpm drizzle-kit generate`, apply migration SQL
2. **DB helpers**: Add query functions in `server/db.ts`
3. **Procedures**: Add tRPC procedures in `server/routers.ts` using `protectedProcedure`
4. **Frontend**: Create page in `client/src/pages/FeaturePage.tsx`
5. **Route**: Add route in `client/src/App.tsx` inside `DashboardRouter`
6. **Navigation**: Add sidebar item in `client/src/components/DashboardLayout.tsx`
7. **Tests**: Add assertions in `server/seraphim.test.ts` or `server/features.test.ts`
8. **Todo**: Update `todo.md`

Modifying an existing feature:

1. Identify the page file in `client/src/pages/`
2. Find the corresponding router namespace in `server/routers.ts`
3. Check DB helpers in `server/db.ts` if persistence is involved
4. Check shared types/constants in `shared/`
5. Run `pnpm test` after changes

16.4 Key Conventions

- **All LLM calls** go through `invokeLLM()` from `server/_core/llm.ts` — never call OpenAI directly
- **All file uploads** go through `storagePut()` from `server/storage.ts` — never write to local filesystem
- **All audit logging** uses `db.addAuditLog()` — every user-facing action should be logged
- **Zod validation** on all procedure inputs — no raw `any` types
- **superjson serialization** — `Date` objects pass through tRPC correctly

- **Markdown rendering** uses `<Streamdown>` component — never `dangerouslySetInnerHTML`
- **Error handling** — LLM calls wrapped in try/catch with user-visible error messages
- **Anonymous fallback** — never assume `ctx.user` requires real authentication

16.5 Files You Should NOT Edit

Path	Reason
<code>server/_core/*</code>	Framework plumbing (OAuth, context, Vite bridge)
<code>client/src/components/ui/*</code>	shadcn/ui components (update via CLI)
<code>patches/*</code>	Wouter routing patch
<code>drizzle/meta/*</code>	Drizzle migration metadata

16.6 Files You SHOULD Edit

Path	Purpose
<code>drizzle/schema.ts</code>	Add/modify database tables
<code>server/db.ts</code>	Add/modify database query helpers
<code>server/routers.ts</code>	Add/modify tRPC procedures
<code>server/eiram.ts</code>	Modify EiRAM analysis engine
<code>shared/modes.ts</code>	Add/modify chat modes
<code>shared/terra.ts</code>	Add/modify Argus Terra types
<code>shared/network-*.ts</code>	Add/modify network knowledge bases
<code>client/src/pages/*.tsx</code>	Add/modify page components
<code>client/src/components/DashboardLayout.tsx</code>	Modify sidebar navigation
<code>client/src/components/TopNav.tsx</code>	Modify top navigation
<code>client/src/components/terra/*.tsx</code>	Modify Argus Terra subcomponents
<code>client/src/App.tsx</code>	Add/modify routes
<code>client/src/index.css</code>	Modify global theme
<code>todo.md</code>	Track features and bugs

16.7 Common Patterns

Adding a new LLM-powered feature:

```
// In server/routers.ts
myFeature: router({
  analyze: protectedProcedure
    .input(z.object({ text: z.string().min(1) }))
    .mutation(async ({ ctx, input }) => {
      try {
        const response = await invokeLLM({
          messages: [
            { role: "system", content: "Your system prompt here" },
            { role: "user", content: input.text },
          ],
        });
        const result = response.choices[0]?.message?.content ?? "Failed";
        await db.addAuditLog(ctx.user.id, "Feature used", "system",
input.text.substring(0, 100));
        return { result };
      } catch (e: any) {
        return { result: `Error: ${e.message}` };
      }
    }),
}),
```

Adding a new page:

```
// In client/src/pages/MyFeaturePage.tsx
import { trpc } from "@lib/trpc";

export default function MyFeaturePage() {
  const mutation = trpc.myFeature.analyze.useMutation();
  // ... UI with loading/error/success states
}
```

16.8 Deployment

The application is deployed on Manus infrastructure. To deploy:

1. Save a checkpoint via Manus UI
2. Click “Publish” in the Manus Management UI

The deployed URL is: <https://seraphimai-gbr265kv.manus.space>

17. DEVELOPMENT WORKFLOW

17.1 Scripts

Script	Command	Purpose
Dev server	<code>pnpm dev</code>	Start development server with hot reload
Build	<code>pnpm build</code>	Production build (Vite + esbuild)
Start	<code>pnpm start</code>	Run production build
Type check	<code>pnpm check</code>	TypeScript type checking
Test	<code>pnpm test</code>	Run Vitest test suite
Format	<code>pnpm format</code>	Prettier formatting
DB migrate	<code>pnpm db:push</code>	Generate and apply migrations

17.2 Build Pipeline

1. **Frontend:** Vite builds React app to `dist/` with Tailwind CSS processing
2. **Backend:** esbuild bundles `server/_core/index.ts` to `dist/index.js` (ESM, external packages)
3. **Production:** `node dist/index.js` serves both API and static frontend

17.3 Version History

Version	Checkpoint	Key Changes
v1.0	Initial	Core 8 modules (Chat, Network, Code, Engineering, Analysis, Memory, Plugins, Audit)
v2.0	—	NSA aesthetic restyle, Pearl design system
v3.0	—	Landing page, Weather, Flights, News, Discover
v4.0	—	Anonymous operator mode, mode selector (12 modes), file upload, EiRAM deep analysis
v5.0	—	Settings, Instagram Intel, Chat Search, SystemSentinel, Network Intelligence
v6.0	—	Command Deck, TopNav, CMIT 265 full toolkit
v7.0	287f50a1	Argus Terra (14 components), Argus Vigil, resizable sidebar, 404 page, chat ownership security
v8.0	f6d4a9ec	Marine Traffic, TerraGlobe Google Maps upgrade, News nav item

APPENDIX A: FULL ROUTE MAP

Route	Component	Layout	Auth Required
/	LandingPage	Public (TopNav only)	No
/deck	CommandDeckPage	Public (TopNav only)	No
/components	ComponentShowcase	Public (TopNav only)	No
/404	NotFound	Public (TopNav only)	No
/chat	ChatPage	Dashboard (sidebar)	Anonymous OK
/network	NetworkPage	Dashboard	Anonymous OK
/argus-vigil	ArgusVigilPage	Dashboard	Anonymous OK
/argus-terra	ArgusTerraPage	Dashboard	Anonymous OK
/argus-terra/session/:id	ArgusTerraPage	Dashboard	Anonymous OK
/code	CodePage	Dashboard	Anonymous OK
/engineering	EngineeringPage	Dashboard	Anonymous OK
/analysis	AnalysisPage	Dashboard	Anonymous OK
/memory	MemoryPage	Dashboard	Anonymous OK

Route	Component	Layout	Auth Required
/plugins	PluginsPage	Dashboard	Anonymous OK
/audit	AuditPage	Dashboard	Anonymous OK
/discover	DiscoverPage	Dashboard	Anonymous OK
/news	NewsPage	Dashboard	Anonymous OK
/weather	WeatherPage	Dashboard	Anonymous OK
/flights	FlightsPage	Dashboard	Anonymous OK
/marine-traffic	MarineTrafficPage	Dashboard	Anonymous OK
/settings	SettingsPage	Dashboard	Anonymous OK
/instagram	InstagramPage	Dashboard	Anonymous OK
/sentinel	SentinelPage	Dashboard	Anonymous OK
/netintel	NetworkIntelPage	Dashboard	Anonymous OK

APPENDIX B: SENTINEL CHECK CATALOG

System Health (10 checks)

Check Name	Script	Description
SFC Scan & Repair	check-sfc-scan.ps1	Scans and repairs Windows system files
DISM Health Check & Restore	check-dism-health.ps1	Checks and restores Windows component store health
CHKDSK with Auto-Repair	check-chkdsk.ps1	Checks disk integrity and repairs errors
Windows Update Audit	check-windows-updates.ps1	Audits pending and installed Windows updates
Driver Integrity Check	check-driver-integrity.ps1	Verifies driver signatures and integrity
Disk Space Check	check-disk-space.ps1	Monitors free disk space across all drives
Memory Usage	check-memory.ps1	Reports current memory utilization
CPU Temperature	check-cpu-temperature.ps1	Reads CPU thermal sensor data
Service Status	check-service-status.ps1	Checks critical Windows service states
Network Connectivity	check-network-connectivity.ps1	Tests network adapter and internet connectivity

Security (5 checks)

Check Name	Script	Description
Startup Program Audit	check-startup-programs.ps1	Lists and audits auto-start programs
Process Watchdog	check-process-watchdog.ps1	Monitors running processes for anomalies
Network Port Monitor	check-network-ports.ps1	Scans open network ports and listeners
Firewall Rule Audit	check-firewall-rules.ps1	Audits firewall rules for risky exceptions
Event Log Criticals	check-event-log-criticals.ps1	Scans Windows event logs for critical errors

Performance (8 checks)

Check Name	Script	Description
Disk Defrag / Optimize	check-disk-defrag.ps1	Checks disk fragmentation and optimization status
Memory Diagnostic	check-memory-diagnostic.ps1	Runs Windows Memory Diagnostic checks
Resource Usage Dashboard	check-resource-usage.ps1	Comprehensive CPU, memory, and disk usage report
Scheduled Task Audit	check-scheduled-tasks.ps1	Audits Windows scheduled tasks for anomalies
Service Status Viewer	check-service-status-viewer.ps1	Detailed service status with dependencies
Disk I/O Performance	check-disk-io.ps1	Measures disk read/write performance
Network Latency	check-network-latency.ps1	Tests network latency to key endpoints
Application Response Time	check-app-response-time.ps1	Measures application startup and response times

Inventory (4 checks)

Check Name	Script	Description
Installed Software List	check-installed-software.ps1	Lists all installed software with versions
Driver List with Versions	check-driver-list.ps1	Enumerates all drivers with version info
Patch History Timeline	check-patch-history.ps1	Shows Windows update and patch history
BSOD Dump Parser	check-bsod-dump.ps1	Parses blue screen crash dump files

Logs (1 check)

Check Name	Script	Description
Session Log Timeline	check-session-log-timeline.ps1	Shows login/logout session timeline

APPENDIX C: CHAT MODE REFERENCE

Mode	System Prompt Extension	Output Format
Standard	Warm, intelligent, direct. Bottom line → Analysis → Evidence → Risks → Recommended move	Free-form prose
EiRAM	Full analytic pipeline. 5 modules (IRI, VDM, ECS, EEM, PFM) + dashboard	Structured dashboard
Legal	IRAC structure. Issue → Rule → Application → Conclusion	Legal brief format
Technical	Systems architecture. Components → Dependencies → Risks → Mitigations	Technical document
Political	Rhetoric analysis. Incentives → Escalation → Power dynamics	Intelligence brief
Behavioral	Personality assessment. Traits → Patterns → Risks → Dynamics	Clinical assessment
Writing	Prose editing. Structure → Clarity → Argument strength → Style	Annotated revision
Mythic	Cinematic, symbolic, poetic. Archetypal analysis	Narrative prose
Homework	Clear, concise, assignment-compliant. Step-by-step	Academic format
Briefing	Executive summary. Key judgments → Confidence → Recommendations	Intel briefing
Red Team	Adversarial thinking. Vulnerabilities → Attack vectors → Counters	Red team report
Dashboard	Compact structured format. KPIs → Status → Actions	Dashboard cards

END OF WHITE PAPER

This document is the authoritative reference for the Seraphim program. It should be provided to any agent, developer, or system taking over development. Keep it updated as the codebase evolves.